BITWISE

CHEAT SHEETS

A Complete Reference for Bit Manipulation, Across Languages

Binary & Hex · Two's Complement · Shifts · Masks · Bit Tricks · Popcount · XOR · Flags · Bitsets · Colors · Endianness · Subnets · Hashing · Python · Java · JavaScript · C / C++ · Go · Rust · CLI

23 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01 Binary, Hex & Octal
02 Signed Integers & Two's Complement
03 AND, OR, XOR & NOT
04 Shifts & Rotations

CORE TECHNIQUES

05 Single-Bit Operations
06 Masks & Bit Fields
07 Lowest Set Bit & Powers of Two
08 Counting & Scanning Bits
09 XOR Tricks

REAL-WORLD USES

10 Flags & Permissions
11 Bitsets & Subset Enumeration
12 Colors & Packed Data
13 Bytes, Endianness & Binary Formats
14 IP Addresses, Subnets & CIDR
15 Hashing, Randomness & Branchless Tricks

LANGUAGE GUIDES

16 Bitwise in Python
17 Bitwise in Java
18 Bitwise in JavaScript
19 Bitwise in C & C++
20 Bitwise in C# & Go
21 Bitwise in Rust, Kotlin, Ruby & SQL
22 Bitwise on the Command Line

QUICK REFERENCE

23 Quick Reference & Language Matrix

BIT

Table of Contents

23 sheets · syntax · examples · notes

FOUNDATIONS 01 – 04

01 Binary, Hex & Octal
02 Signed Integers & Two's…
03 AND, OR, XOR & NOT
04 Shifts & Rotations

CORE TECHNIQUES 05 – 09

05 Single-Bit Operations
06 Masks & Bit Fields
07 Lowest Set Bit & Power…
08 Counting & Scanning Bi…
09 XOR Tricks

REAL-WORLD USES 10 – 15

10 Flags & Permissions
11 Bitsets & Subset Enum…
12 Colors & Packed Data
13 Bytes, Endianness & Bi…
14 IP Addresses, Subnets …
15 Hashing, Randomness …

LANGUAGE GUIDES 16 – 22

16 Bitwise in Python
17 Bitwise in Java
18 Bitwise in JavaScript
19 Bitwise in C & C++
20 Bitwise in C# & Go
21 Bitwise in Rust, Kotlin, R…
22 Bitwise on the Comma…

QUICK REFERENCE 23

23 Quick Reference & Lan…

Binary, Hex & Octal

positional notation · bit / nibble / byte / word · `0b` / `0x` / `0o` · hex ↔ binary · MSB / LSB · _ separators

BITS ARE PLACE VALUES

Computers store integers as **bits**, and each bit is worth a power of two. Binary is the native form. **Hexadecimal** packs 4 bits (a nibble) into one digit and **octal** packs 3, so both are shorthand for bit patterns. Bits are numbered from 0 at the right (the **LSB**, least significant bit) up to w - 1 at the left (the **MSB**).

VOCABULARY & LITERALS

- nibble = 4 bits · byte = 8 · word = 32 or 64
- `0b1010` binary · `0x1F` hex · `0o17` octal
- Separators: `0b1011_0010` (C++: `0b1011'0010`)

READING & CONVERTING BINARY (PSEUDOCODE)

```
// Place values for 8 bits: bit 7 … bit 0
//  128  64  32  16   8   4   2   1
//    1   0   1   1   0   0   1   0     = 0b1011_0010
//  128    + 32 +16         + 2         = 178 = 0xB2

// Decimal → binary: repeated division by 2
function toBinary(n)                  // n > 0
    s ← ""
    while n > 0
        s ← str(n mod 2) + s          // remainder = next bit
        n ← n div 2                   // drop that bit
    return s

// Binary → decimal: shift-and-add
function fromBinary(s)
    n ← 0
    for each ch in s
        n ← n * 2 + digit(ch)
    return n
```

CODE EXPLAINED

→ `n mod 2` — the lowest bit; `n div 2` drops it (same as `n >> 1`)

→ Hex: group bits in 4s from the right. `1011 0010` → `B 2` → `0xB2`

→ Octal: group in 3s. `10 110 010` → `2 6 2` → `0o262`

→ Leading zeros show the width, not the value: `0b0000_0101` is still 5

→ Every language has built-ins for this (see Sheets 16–22); the loop shows what they do

THE 16 NIBBLES

DEC	BIN	HEX	DEC	BIN	HEX
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

POWERS OF TWO

2^n	VALUE
2^8	256 (one byte)
2^{10}	1,024 (≈ 1 K)
2^{16}	65,536
2^{31}	2,147,483,648
2^{32}	4,294,967,296
2^{53}	≈ 9.0 × 10^{15} (JS safe-int limit)
2^{64}	≈ 1.8 × 10^{19}

LITERAL SYNTAX

LANGUAGE	BIN	HEX	OCT
Python	`0b101`	`0x1F`	`0o17`
JavaScript	`0b101`	`0x1F`	`0o17`
Java	`0b101`	`0x1F`	`017`
C / C++	`0b101` [1]	`0x1F`	`017`
C#	`0b101`	`0x1F`	—
Go	`0b101`	`0x1F`	`0o17`
Rust	`0b101`	`0x1F`	`0o17`
bash	`2#101`	`0x1F`	`017`

⚠ COMMON MISTAKES

- Counting bit 0 from the left. Bit 0 is the rightmost (LSB)
- `017` is 15, not 17: a leading zero means octal in C, Java and bash
- Forgetting one hex digit is exactly 4 bits (`0xFF` = 8 ones)

✓ TIPS & BEST PRACTICES

- Learn 0–F in binary by heart; hex masks become readable
- Write masks in hex (`0xFF00`), not decimal (`65280`)
- [1] `0b` arrived in C23 (GCC/Clang extension before) and C++14

QUICK SUMMARY

- `0b` / `0x` / `0o` — binary, hex, octal literals
- bit i — worth 2^i, counted from the right starting at 0
- 1 hex digit = 4 bits; 2 hex digits = 1 byte
- `n mod 2`, `n div 2` — read, then drop, the lowest bit

Sheet 02

Signed Integers & Two's Complement

sign bit · −x = ~x + 1 · ranges · overflow · sign extension · MIN

HOW NEGATIVE NUMBERS ARE STORED

Fixed-width integers are either **unsigned** (every bit adds value) or **signed**. Almost every machine stores signed integers in **two's complement**: the top bit has a negative weight of -2^(w-1), so counting past the largest positive value wraps to the most negative one. Negation is ~x + 1, and the same adder circuit handles signed and unsigned math.

KEY FACTS

- Sign bit = MSB. 1 means negative
- `-1` is all ones: `0xFF`, `0xFFFFFFFF`
- `MIN` has no positive twin: -(-128) wraps to -128
- C23 and C++20 now require two's complement

8-BIT TWO'S COMPLEMENT (PSEUDOCODE)

```
// bit 7 weighs -128; bits 6..0 weigh 64..1 as usual
//   bits         unsigned   signed
//   0000_0000        0         0
//   0000_0001        1         1
//   0111_1111      127       127    ← signed MAX
//   1000_0000      128      -128    ← signed MIN
//   1111_1110      254        -2
//   1111_1111      255        -1

// Negate 5:
//   5        = 0000_0101
//   ~5       = 1111_1010
//   ~5 + 1   = 1111_1011    = -5 signed, 251 unsigned

// Read a w-bit unsigned pattern as signed
function toSigned(u, w)                 // 0 ≤ u < 2^w
    if u ≥ 2^(w - 1)                    // sign bit set
        return u - 2^w
    return u
```

CODE EXPLAINED

→ `u - 2^w` — 251 - 256 = -5. Same bits, two readings

→ Overflow: 127 + 1 = -128 in 8-bit signed (it wraps)

→ Unsigned wraps too: 0 - 1 = 255 in 8-bit

→ **Sign extension**: widening -5 to 16 bits copies the sign bit: `1111_1111_1111_1011`

→ **Zero extension** (unsigned): 251 → `0000_0000_1111_1011`

RANGES BY WIDTH

BITS	UNSIGNED	SIGNED
8	0 … 255	-128 … 127
16	0 … 65,535	-32,768 … 32,767
32	0 … 4,294,967,295	-2,147,483,648 … 2,147,483,647
64	0 … ≈ 1.8×10^{19}	≈ ±9.2×10^{18}

OTHER SIGNED ENCODINGS

ENCODING	RULE	WHERE YOU MEET IT
Two's complement	-x = ~x + 1, one zero	every modern CPU's integers
One's complement	-x = ~x, has +0 and -0	old machines, Internet checksum
Sign-magnitude	sign bit + magnitude	IEEE-754 floats (sign bit)
Offset / excess-K	stored = value + K	float exponents (bias 127)

⚠ COMMON MISTAKES

- `abs(INT_MIN)` is still `INT_MIN` (overflow). Check before negating
- C: `-1 < 1u` is false (-1 becomes `0xFFFFFFFF`)
- Byte `0xF0` read as signed is -16, not 240. Use `& 0xFF`
- Assuming overflow traps. Most languages wrap silently

✓ TIPS & BEST PRACTICES

- Use unsigned types for bit patterns where you can
- `~0` is a ready-made all-ones mask
- Sign-extend a w-bit field: `(x ^ m) - m`, `m = 1 << (w-1)`
- Landmarks: `0x7FFFFFFF` = INT_MAX, `0x80000000` = INT_MIN

QUICK SUMMARY

- two's complement — top bit weighs -2^(w-1)
- `-x = ~x + 1` — invert, then add one
- `-1` — every bit set
- sign extension — widen by copying the sign bit
- `MIN` — has no positive counterpart

AND, OR, XOR & NOT

FOUR OPERATORS, ONE BIT AT A TIME

The bitwise operators work on every bit position independently, like 32 or 64 tiny logic gates side by side. **AND** keeps bits that are set in both inputs, **OR** keeps bits set in either, **XOR** keeps bits that differ, and **NOT** flips every bit. Every other trick in this set is built from these four plus shifts.

NOTES

- `&` `|` `^` `~` — the C-style symbols nearly every language uses
- `&&` / `||` are logical and short-circuit; bitwise ops always evaluate both sides
- Signed: `~x = -x - 1`, so `~5 = -6`

WORKED 8-BIT EXAMPLES (PSEUDOCODE)

```
a ← 0b1100_1010              // 202
b ← 0b1010_0110              // 166

//     1100_1010   a
//   & 1010_0110   b
//   = 1000_0010   130    AND: 1 only where both are 1
//
//     1100_1010
//   | 1010_0110
//   = 1110_1110   238    OR: 1 where either is 1
//
//     1100_1010
//   ^ 1010_0110
//   = 0110_1100   108    XOR: 1 where they differ
//
//   ~ 1100_1010
//   = 0011_0101    53    NOT (8-bit unsigned view)

if (x & 1) = 0               // keep the parentheses
    print("even")
```

CODE EXPLAINED

- → **AND** clears bits. Use it to keep only the bits in a mask
- → **OR** sets bits. Use it to turn on the bits in a mask
- → **XOR** flips bits. Use it to toggle, or to find what changed
- → **NOT** inverts all w bits, including the sign bit
- → C, C++, Java, JS, C#: `x & 1 == 0` parses as `x & (1 == 0)`. Python, Go and Rust bind `&` tighter, so it works there

TRUTH TABLE

A	B	A & B	A | B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

`~0 = 1` `~1 = 0`

IDENTITIES

EXPRESSION	EQUALS
`x & 0`	`0`
`x & ~0`	`x`
`x | 0`	`x`
`x | ~0`	`~0 (all ones)`
`x ^ 0`	`x`
`x ^ x`	`0`
`x ^ ~0`	`~x`
`x & x, x | x`	`x`

LAWS WORTH KNOWING

LAW	FORM
De Morgan	`~(a & b) = ~a | ~b`
De Morgan	`~(a | b) = ~a & ~b`
Distribute	`a & (b | c) = (a & b) | (a & c)`
XOR from AND/OR	`a ^ b = (a | b) & ~(a & b)`
Addition	`a + b = (a ^ b) + 2(a & b)`

⚠ COMMON MISTAKES

- Using `&` as logical AND on ints: `1 & 2` is 0 (false) while `1 && 2` is true
- Leaving out parentheses: `x & MASK == 0` in C, Java or JS
- Expecting `~5` to be 2 (3-bit thinking). On a signed int it's -6
- Reading `^` as "to the power of". It's XOR; use `pow()` for powers

✓ TIPS & BEST PRACTICES

- Read `&` as "keep", `|` as "set", `^` as "flip"
- XOR undoes itself: `(a ^ k) ^ k = a`
- Parenthesize every bitwise subexpression inside a comparison
- Use De Morgan to turn "clear these bits" into `x & ~mask`

QUICK SUMMARY

- `&` — 1 if both bits are 1
- `|` — 1 if either bit is 1
- `^` — 1 if the bits differ
- `~` — flip every bit (`~x = -x - 1` signed)

Shifts & Rotations

SLIDING BITS LEFT AND RIGHT

Shifts move every bit n positions. **Left shift** fills with zeros and multiplies by 2^n (bits pushed off the top are lost). **Right shift** divides by 2^n and comes in two kinds: **arithmetic** copies the sign bit (for signed numbers) and **logical** fills with zeros (for unsigned). A **rotate** wraps the bits that fall off one end back in at the other.

NOTES

- `x >> n` = floor(x / 2^n). It rounds toward −∞: `-7 >> 1` is -4, but `-7 / 2` is -3
- `>>>` exists in Java, JS, C# 11+ and Kotlin (`ushr`). C, C++, Go and Rust shift unsigned types instead
- Shifting by ≥ the width is undefined in C/C++ and masked in Java/JS/C#

SHIFTS IN 8 BITS (PSEUDOCODE)

```
x ← 0b0001_0110              // 22
x << 1       // 0b0010_1100     44   (× 2)
x << 3       // 0b1011_0000    176   (× 8)
x >> 2       // 0b0000_0101      5   (22 div 4)

// 8-bit signed -16 = 1111_0000
-16 >> 2     // 1111_1100 = -4    arithmetic: sign bit copied
-16 >>> 2    // 0011_1100 = 60    logical: zeros shifted in

// Rotate left by n inside a w-bit unsigned value
function rotl(x, n, w)
    n ← n mod w
    if n = 0
        return x                        // avoid shifting by w
    mask ← (1 << w) - 1
    return ((x << n) | (x >>> (w - n))) & mask

rotl(0b1001_0110, 3, 8)      // 0b1011_0100
```

CODE EXPLAINED

- → Shifting into the sign bit flips the sign: `1 << 31` is negative in a 32-bit int
- → Real languages shift 32 or 64 bits: `-16 >>> 2` on a 32-bit int is 1,073,741,820
- → The `n = 0` guard avoids a shift by w, which is undefined or masked
- → Rotate right is `rotl(x, w - n, w)`
- → Compilers turn this pattern into one ROL/ROR instruction

SHIFT BEHAVIOR BY LANGUAGE

LANGUAGE	>> ON SIGNED	>>>	COUNT ≥ WIDTH
C	impl.-defined (arithmetic in practice)	—	undefined
C++20	arithmetic	—	undefined
Java	arithmetic	yes	masked: & 31 / & 63
JavaScript	arithmetic, 32-bit	yes	masked: & 31
C#	arithmetic	C# 11+	masked
Go	arithmetic	— (use uint)	0, or -1 for negative >>
Rust	arithmetic	— (use u32)	panics in debug
Python	arithmetic (floor)	—	no limit (big ints)

SHIFTS AS ARITHMETIC

EXPRESSION	EQUALS
`1 << n`	2^n
`x << n`	x × 2^n (until overflow)
`x >> n`	floor(x / 2^n)
`x & ((1 << n) - 1)`	x mod 2^n (x ≥ 0)
`(x + (1 << n) - 1) >> n`	ceil(x / 2^n) (x ≥ 0)
`x >> (w - 1)`	0 or −1: sign of x

⚠ COMMON MISTAKES

- `1 << 40` with a 32-bit 1. Java masks the count to 8 and gives 256. Write `1L << 40`
- Expecting `>>` to divide negatives like `/`: `-7 >> 1` is -4, `-7 / 2` is -3
- Precedence: `1 << n - 1` means `1 << (n - 1)`, since shifts bind looser than + and -
- Using `>>>` on a Java `byte`: it's promoted to int first, so sign bits leak in

✓ TIPS & BEST PRACTICES

- Write `x * 8`, not `x << 3`, unless you mean bits. The compiler optimizes either
- Use unsigned types (or `>>>`) for bit patterns so right shifts never sign-fill
- Prefer built-in rotates: `Integer.rotateLeft`, `bits.RotateLeft`, `std::rotl`
- Parenthesize: `(1 << n) - 1`

QUICK SUMMARY

- `<<` — shift left: × 2^n, zeros in
- `>>` — arithmetic right: sign bit in
- `>>>` — logical right: zeros in
- rotate — bits shifted out wrap around
- count ≥ width — undefined or masked: avoid

Single-Bit Operations

TEST, SET, CLEAR, TOGGLE

Most bit work comes down to four moves on one position i: **test**, **set**, **clear** and **toggle**. Each builds a one-bit mask with `1 << i`, then combines it with x using AND, OR, AND-NOT or XOR. Learn these four lines and you can read most flag and bitset code.

NOTES

- Positions count from 0 at the LSB
- `1 << i` must be as wide as x. For i ≥ 31 use a 64-bit one: `1L << i`
- Set/clear/toggle return a new value; assign it back (`x |= m`)

THE FOUR MOVES (PSEUDOCODE)

```
x ← 0b0010_1001                    // 41: bits 0, 3, 5 set
i ← 3

bit   ← (x >> i) & 1               // test → 1
isSet ← (x & (1 << i)) ≠ 0         // test as boolean → true

x | (1 << 2)       // 0b0010_1101   set bit 2
x & ~(1 << 3)      // 0b0010_0001   clear bit 3
x ^ (1 << 5)       // 0b0000_1001   toggle bit 5

// Write bit i to v (0 or 1) without branching
function setBitTo(x, i, v)
    return (x & ~(1 << i)) | (v << i)

isOdd ← (x & 1) = 1                // lowest bit: odd/even

// Several bits at once with one mask
m      ← (1 << 0) | (1 << 3)       // 0b0000_1001
allSet ← (x & m) = m               // true: both bits set
anySet ← (x & m) ≠ 0               // true: at least one
```

CODE EXPLAINED

- → Test with `≠ 0`, not `= 1`: `x & (1 << 3)` is 8, not 1
- → `~(1 << i)` is all ones except bit i: an AND mask with one hole
- → `setBitTo` clears first, then ORs v in, so it works whatever bit i held
- → `x & 1` handles negatives better than `x mod 2`: in C/Java `-3 % 2` is -1, `-3 & 1` is 1

THE FOUR MOVES

MOVE	EXPRESSION	COMPOUND FORM
Test	`(x >> i) & 1`	—
Set	`x | (1 << i)`	`x |= 1 << i`
Clear	`x & ~(1 << i)`	`x &= ~(1 << i)`
Toggle	`x ^ (1 << i)`	`x ^= 1 << i`
Write v	`(x & ~(1 << i)) | (v << i)`	—

TESTING X = 41

I	1 << I	BIT
0	0b0000_0001	1
1	0b0000_0010	0
2	0b0000_0100	0
3	0b0000_1000	1
4	0b0001_0000	0
5	0b0010_0000	1

A 64-BIT "1" BY LANGUAGE

LANGUAGE	EXPRESSION
C / C++	`1ULL << i`
Java	`1L << i`
JavaScript	`1n << BigInt(i)`
C#	`1UL << i`
Go	`uint64(1) << i`
Rust	`1u64 << i`
Python	`1 << i (any size)`

⚠ COMMON MISTAKES

- `(x & (1 << i)) == 1` is only true for i = 0. Compare with `!= 0`
- `1 << 31` on a 32-bit signed int is negative. Use unsigned
- `x | 1 << i` on its own line changes nothing. Assign it
- `~1 << i` is `(~1) << i`. Write `~(1 << i)`

✓ TIPS & BEST PRACTICES

- Name positions or masks: `READ = 1 << 0`
- Use library helpers: `BitSet`, `std::bitset::test/set/reset/flip`
- Group bits in one mask to test several at once
- Prefer `!= 0` tests; they work for any bit position

QUICK SUMMARY

- `(x >> i) & 1` — test bit i
- `x | (1 << i)` — set bit i
- `x & ~(1 << i)` — clear bit i
- `x ^ (1 << i)` — toggle bit i

Masks & Bit Fields

`(1 << n) - 1` · range mask · extract `(x >> lo) & mask` · insert field · clear a range · `~0` · named masks

MASKS SELECT BITS; FIELDS STORE NUMBERS

A **mask** is a value whose 1 bits mark the positions you care about: AND with it keeps those bits, OR sets them, AND-NOT clears them. A **bit field** is a run of adjacent bits holding a small number inside a bigger word. Read a field with **shift, then mask**; write it with **clear the slot, then OR**.

NOTES

- `(1 << n) - 1` = n low ones: n = 4 → `0b1111`
- n = w overflows. Use `~0` for all bits
- Name masks: `LOW_NIBBLE = 0x0F`

BUILD MASKS, READ & WRITE FIELDS (PSEUDOCODE)

```
low(n)        ← (1 << n) - 1                // n low ones
range(lo, hi) ← low(hi - lo + 1) << lo      // ones in bits lo..hi
low(4)        // 0b0000_1111
range(2, 5)   // 0b0011_1100

function getField(x, lo, width)       // shift down, then mask
    return (x >> lo) & low(width)

function setField(x, lo, width, v)    // clear slot, then OR in
    m ← low(width) << lo
    return (x & ~m) | ((v << lo) & m)

x ← 0b1101_0110
getField(x, 2, 3)          // 0b101 = 5   (bits 2..4)
setField(x, 2, 3, 0b010)   // 0b1100_1010
```

CODE EXPLAINED

- → `& m` after shifting v stops a too-wide value from spilling into neighbors
- → Mask-then-shift `(x & m) >> lo` works too, but needs the shifted mask
- → `range(lo, hi)` is inclusive at both ends: width = hi - lo + 1
- → Fields don't need byte alignment; any lo and width work

MASKS TO RECOGNIZE

MASK	SELECTS
`0x0F / 0xF0`	low / high nibble
`0xFF`	low byte
`0x7FFFFFFF`	all but the sign bit (32-bit)
`0x55555555`	even bits (0101…)
`0xAAAAAAAA`	odd bits (1010…)

RANGE OPERATIONS

GOAL	EXPRESSION
keep bits lo..hi	`x & range(lo, hi)`
clear bits lo..hi	`x & ~range(lo, hi)`
set bits lo..hi	`x | range(lo, hi)`
flip bits lo..hi	`x ^ range(lo, hi)`
sign-extend field f	`(f ^ m) - m, m = 1 << (width-1)`

REAL LAYOUT: FAT/DOS DATE & TIME (TWO 16-BIT WORDS) — EXAMPLE 2026-10-10 14:35:20

FIELD	BITS	WIDTH	READ	EXAMPLE
year - 1980	date 15–9	7	`(d >> 9) & 0x7F`	46 → 2026
month	date 8–5	4	`(d >> 5) & 0x0F`	10
day	date 4–0	5	`d & 0x1F`	10 (date word = 0x5D4A)
hour	time 15–11	5	`(t >> 11) & 0x1F`	14
minute	time 10–5	6	`(t >> 5) & 0x3F`	35
second ÷ 2	time 4–0	5	`t & 0x1F`	10 → 20 s (2-second resolution)

⚠ COMMON MISTAKES

- `x & m >> lo` is `x & (m >> lo)` in C-family. Parenthesize
- OR-ing a new value without clearing the slot merges old and new bits
- `(1 << 32) - 1` on a 32-bit int shifts by the width. Use `0xFFFFFFFF`

✓ TIPS & BEST PRACTICES

- Define `SHIFT` and `MASK` constants per field and use them as a pair
- Keep masks unsigned so shifts never sign-fill
- Struct bit fields (Sheet 19) aren't portable; mask by hand for file formats

QUICK SUMMARY

- `(1 << n) - 1` — n low ones
- `(x >> lo) & mask` — read a field
- `(x & ~m) | (v << lo)` — write a field
- `x & m` / `x | m` / `x & ~m` — keep / set / clear

Sheet 07

Lowest Set Bit & Powers of Two

`x & (x − 1)` · `x & −x` · `x | (x + 1)` · power-of-two test · next power of two · `x & (2ⁿ − 1)` · align up / down

TRICKS BUILT ON ±1

Subtracting 1 flips the lowest set bit and every zero below it, so `x & (x − 1)` erases the lowest 1. In two's complement `−x = ~x + 1`, so `x & −x` keeps **only** the lowest 1. A power of two has exactly one bit set, which makes it easy to test for, round up to, and use for fast modulo and alignment.

NOTES

- Assume x is unsigned or non-negative unless noted
- Without unary minus on unsigned: `x & (~x + 1)`
- 0 is not a power of two; check `x ≠ 0`

THE ±1 FAMILY (PSEUDOCODE)

```
x ← 0b0101_1000                  // 88

x & (x - 1)     // 0b0101_0000      clear lowest set bit
x & -x          // 0b0000_1000      isolate lowest set bit (8)
x | (x - 1)     // 0b0101_1111      fill trailing zeros
x | (x + 1)     // 0b0101_1001      set lowest zero bit
~x & (x + 1)    // 0b0000_0001      isolate lowest zero bit

isPow2(x) ← x ≠ 0 and (x & (x - 1)) = 0

function nextPow2(x)             // smallest 2^k ≥ x, x ≥ 1
    x ← x - 1
    s ← 1
    while s < 32                 // smear the top 1 rightward
        x ← x | (x >> s)
        s ← s * 2
    return x + 1
```

CODE EXPLAINED

- → `x & (x - 1)` drives Kernighan's popcount and set-bit loops (Sheets 08, 11)
- → `x & -x` is the Fenwick-tree step: `i ← i + (i & -i)`
- → Smearing copies the top 1 into every lower bit, giving $2^k - 1$; +1 finishes
- → The opening `x - 1` keeps exact powers: `nextPow2(64)` = 64

MODULO & ALIGNMENT (2ⁿ ONLY)

GOAL	EXPRESSION
x mod 2^n	`x & (2ⁿ - 1)`
x div 2^n	`x >> n`
multiple of 2^n?	`(x & (2ⁿ - 1)) = 0`
align down to a	`x & ~(a - 1)`
align up to a	`(x + a - 1) & ~(a - 1)`

WHERE YOU SEE THESE

TRICK	USED IN
`x & (x - 1)`	popcount loops, power-of-two checks
`x & -x`	Fenwick trees, lowest-priority flag
`nextPow2`	hash table capacity (Java `tableSizeFor`)
`align up`	allocators, 4096-byte pages, struct padding
`x & (2ⁿ - 1)`	ring buffer indexes, hash buckets

WORKED VALUES

X	ISPOW2	NEXTPOW2	X & 7 (MOD 8)	ALIGN UP TO 8	ALIGN DOWN TO 8	X & -X
5	false	8	5	8	0	1
64	true	64	0	64	64	64
100	false	128	4	104	96	4
1000	false	1024	0	1000	1000	8

⚠ COMMON MISTAKES

- `isPow2(0)` is true without the `x ≠ 0` check
- Rust debug panics on `-x` overflow; use `x & x.wrapping_neg()`
- `-3 % 8` is -3 in C/Java, but `-3 & 7` is 5

✓ TIPS & BEST PRACTICES

- C++20: `std::has_single_bit`, `std::bit_ceil`, `std::bit_floor`
- Java: `Integer.highestOneBit`, `Integer.lowestOneBit`
- Size hash tables as 2^n so `h & (cap - 1)` replaces `h % cap`

QUICK SUMMARY

- `x & (x - 1)` — clear lowest 1
- `x & -x` — isolate lowest 1
- `x & (x - 1) = 0`, x ≠ 0 — power of two
- `(x + a - 1) & ~(a - 1)` — align up

Counting & Scanning Bits

popcount · Kernighan · SWAR · trailing zeros (ctz) · leading zeros (clz) · bit length · parity · POPCNT / LZCNT

FOUR QUESTIONS ABOUT A WORD

How many bits are set (**popcount**)? How many zeros sit below the lowest 1 (**ctz**, trailing zeros)? How many zeros sit above the highest 1 (**clz**, leading zeros)? How many bits does the number need (**bit length**)? Modern CPUs answer each in one instruction, and every major language exposes them. The loops below show what the built-ins do.

RELATIONSHIPS

- bit length = w − clz(x) = floor($\log_2$ x) + 1
- ctz(x) = index of the lowest set bit
- parity = popcount(x) mod 2

COUNTING WITHOUT BUILT-INS (PSEUDOCODE)

```
// Kernighan: one iteration per SET bit
function popcount(x)                    // x unsigned
    count ← 0
    while x ≠ 0
        x ← x & (x - 1)                 // clear lowest 1
        count ← count + 1
    return count

// SWAR: all 32 bits in parallel, no loop
function popcount32(x)
    x ← x - ((x >> 1) & 0x55555555)                  // 2-bit sums
    x ← (x & 0x33333333) + ((x >> 2) & 0x33333333)   // 4-bit sums
    x ← (x + (x >> 4)) & 0x0F0F0F0F                  // 8-bit sums
    return ((x * 0x01010101) & 0xFFFFFFFF) >> 24     // add 4 bytes

ctz(x) ← popcount((x & -x) - 1)         // x ≠ 0
```

CODE EXPLAINED

→ Kernighan runs once per set bit: fast on sparse words

→ SWAR adds neighbors in widening groups (1 → 2 → 4 → 8 bits)

→ The multiply sums all four byte counts into the top byte

→ `(x & -x) - 1` turns the trailing zeros into ones, then counts them

BUILT-INS BY LANGUAGE

LANGUAGE	POPCOUNT	TRAILING ZEROS	LEADING ZEROS / LENGTH
C++20 <bit>	std::popcount	std::countr_zero	std::countl_zero / std::bit_width
C (GCC/Clang)	__builtin_popcount	__builtin_ctz	__builtin_clz (undefined at 0)
C23 <stdbit.h>	stdc_count_ones	stdc_trailing_zeros	stdc_leading_zeros / stdc_bit_width
Java	Integer.bitCount	numberOfTrailingZeros	numberOfLeadingZeros
JavaScript	— (loop)	31 - Math.clz32(x & -x)	Math.clz32 (32-bit)
Python	x.bit_count() (3.10+)	(x & -x).bit_length() - 1	x.bit_length()
C#	BitOperations.PopCount	TrailingZeroCount	LeadingZeroCount / Log2
Go	bits.OnesCount	bits.TrailingZeros	bits.LeadingZeros / bits.Len
Rust	x.count_ones()	x.trailing_zeros()	x.leading_zeros() / x.ilog2()

WORKED: X = 44 = 0B0010_1100 (32-BIT)

POPCOUNT	CTZ	CLZ	BIT LENGTH	FLOOR($\log_2$ X)	PARITY	LOWEST 1 (X & -X)
3	2	26	6	5	1 (odd)	4

⚠ COMMON MISTAKES

- `__builtin_clz(0)` is undefined. Check for 0 first
- Python counts the magnitude: `(-1).bit_count()` is 1. Mask first
- Using floating `log2()` for bit length; it rounds wrong for huge values

✓ TIPS & BEST PRACTICES

- CPU support: x86 POPCNT, LZCNT, TZCNT; ARM CLZ, CNT
- Hamming distance = `popcount(a ^ b)`
- Popcount a big bitset word by word and sum

QUICK SUMMARY

- popcount — number of 1 bits
- ctz — index of the lowest 1
- clz — zeros above the highest 1
- bit length — `w - clz(x)`

XOR Tricks

XOR UNDOES ITSELF

XOR is addition without carry, so it is **its own inverse**: `x ^ x = 0` and `x ^ 0 = x`. It's also commutative and associative, so XOR-ing a whole list cancels every value that appears an even number of times. The same property powers toggling, parity, simple checksums and the "what changed?" test.

NOTES

- `a ^ b ^ a = b`, in any order
- `a ^ b = 0` exactly when `a = b`
- `(a ^ b) < 0` — signed ints with different signs

CLASSIC XOR PROBLEMS (PSEUDOCODE)

```
// Single number: every value appears twice except one
function single(nums)
    r ← 0
    for each n in nums
        r ← r ^ n                  // pairs cancel to 0
    return r
single([4, 1, 2, 1, 2])            // 4
// Missing number in 0..n (array has n values)
function missing(nums)
    r ← length(nums)
    for i ← 0 to length(nums) - 1
        r ← r ^ i ^ nums[i]
    return r
missing([3, 0, 1])                  // 2
// Two singles: everything else appears twice
function twoSingles(nums)
    d ← 0
    for each n in nums
        d ← d ^ n                  // d = p ^ q, nonzero
    bit ← d & -d                   // a bit where p and q differ
    p ← 0
    for each n in nums
        if (n & bit) ≠ 0
            p ← p ^ n              // XOR of one group = p
    return (p, d ^ p)

changed ← old ^ new                // 1s = bits that changed
```

CODE EXPLAINED

- → `missing` XORs every index 0..n with every value; whatever is present cancels
- → Toggle between a and b with no branch: `x ← (a ^ b) ^ x`
- → XOR swap (`a ^= b; b ^= a; a ^= b`) zeroes a value swapped with itself. Use a temp
- → `twoSingles` splits the list by one differing bit, so each group hides one single

XOR OF 1..N

N MOD 4	1 ^ 2 ^ … ^ N
0	n
1	1
2	n + 1
3	0

PROPERTIES

RULE	NAME
`x ^ 0 = x`	identity
`x ^ x = 0`	self-inverse
`a ^ b = b ^ a`	commutative
`(a^b)^c = a^(b^c)`	associative
`x ^ ~0 = ~x`	NOT via XOR

WHERE XOR SHOWS UP

USE	HOW
RAID-5 parity	lost disk = XOR of the rest
Gray code	`x ^ (x >> 1)` (Sheet 15)
Hash mixing	`h ^ (h >> 16)`
Stream ciphers	data ^ keystream
Hamming distance	`popcount(a ^ b)`

⚠ COMMON MISTAKES

- XOR-swapping `a[i]` with `a[j]` when i = j sets it to 0
- Treating XOR with a fixed key as encryption; repeating keys are easy to break
- `a ^ b == 0` in C/Java is `a ^ (b == 0)`. Parenthesize

✓ TIPS & BEST PRACTICES

- XOR-fold to find the odd one out in O(1) memory
- Diff two flag words with `old ^ new` before reacting
- For parity, `popcount(x) & 1` beats an XOR loop

QUICK SUMMARY

- `x ^ x = 0`, `x ^ 0 = x` — cancel and keep
- fold a list — pairs cancel
- `a ^ b` — the bits that differ
- `x ^ k ^ k = x` — undo

Flags & Permissions

MANY YES/NO OPTIONS IN ONE INTEGER

A **flag word** packs many on/off options into one integer, one bit per option. Combine options with |, test with &, and remove with & ~. Unix file permissions, OS open flags, regex options, window styles and feature switches all use this pattern: it's compact, fast to test, and travels as a single argument.

NOTES

- Each flag is a distinct power of two: `1 << n`
- A combination is just an int: `READ | WRITE`
- Unix mode = 3 groups of `rwx` bits: user, group, other

FLAGS IN ACTION (PSEUDOCODE)

```
READ    ← 1 << 0                    // 0b001
WRITE   ← 1 << 1                    // 0b010
EXECUTE ← 1 << 2                    // 0b100

perms ← READ | WRITE                // combine:  0b011
(perms & WRITE) ≠ 0                 // has WRITE?  true
both  ← READ | EXECUTE
(perms & both) = both               // has both?   false
perms ← perms | EXECUTE             // add:      0b111
perms ← perms & ~WRITE              // remove:   0b101

mode  ← 0o754                       // rwx r-x r--
user  ← (mode >> 6) & 0o7           // 7 = rwx
group ← (mode >> 3) & 0o7           // 5 = r-x
other ← mode & 0o7                  // 4 = r--
```

CODE EXPLAINED

- → One octal digit = 3 bits = one `rwx` triple; that's why modes are written in octal
- → `& ~WRITE` removes the flag whether or not it was set
- → `umask 022`: new file mode = `0o666 & ~0o022` = `0o644`
- → Special bits sit above: setuid `0o4000`, setgid `0o2000`, sticky `0o1000`

OCTAL DIGIT → RWX

OCTAL	BITS	RWX
7	111	rwx
6	110	rw-
5	101	r-x
4	100	r--
2	010	-w-
0	000	---

COMMON MODES

MODE	SYMBOLIC	USE
755	rwxr-xr-x	programs, folders
644	rw-r--r--	regular files
600	rw-------	private keys
700	rwx------	private folders
1777	rwxrwxrwt	/tmp (sticky)
777	rwxrwxrwx	everyone: avoid

FLAGS IN THE WILD

API	EXAMPLE
POSIX open()	O_WRONLY | O_CREAT | O_TRUNC
Python re	re.IGNORECASE | re.MULTILINE
Java regex	CASE_INSENSITIVE | MULTILINE
C# [Flags]	FileAttributes.Hidden | ReadOnly
Win32	WS_OVERLAPPED | WS_VISIBLE
JS regex	a string ("gi"), not bits

ASKING QUESTIONS OF A FLAG WORD (F = READ | EXECUTE = 0B101)

QUESTION	EXPRESSION	RESULT
has READ?	(f & READ) != 0	true
has READ and EXECUTE?	(f & (READ | EXECUTE)) == (READ | EXECUTE)	true
has READ or WRITE?	(f & (READ | WRITE)) != 0	true
exactly one flag set?	f != 0 && (f & (f - 1)) == 0	false (two set)

⚠ COMMON MISTAKES

- Numbering flags 1, 2, 3, 4: 3 equals `1 | 2`, so flags collide
- `(perms & WRITE) == true`: the result is 2, not a boolean
- `chmod(path, 755)` in code is decimal (= `0o1363`). Write `0o755`

✓ TIPS & BEST PRACTICES

- Use a typed flags type: `enum.Flag`, `[Flags]`, `EnumSet`
- Never renumber flags that are saved to disk or sent over a wire
- Validate input: `(flags & ~ALL) == 0`

QUICK SUMMARY

- `1 << n` — one flag per bit
- `a | b` — combine; `f & ~X` — remove
- `(f & X) != 0` — has X
- `0o755` — `rwxr-xr-x`

Sheet 11

Bitsets & Subset Enumeration

integer as a set · union / intersection · iterate set bits · all 2^n subsets · submasks `(s - 1) & m` · Gosper's hack · bitmask DP

AN INTEGER CAN BE A SET

Let bit i mean "element i is in the set". With up to 64 elements, union, intersection and membership each become one instruction, and counting from 0 to $2^n - 1$ visits **every subset**. This is the engine behind bitmask dynamic programming, and, with arrays of words, behind bitmap indexes, Bloom filters and bit arrays.

NOTES

- Map elements to 0..n-1 first
- 2^n grows fast: n = 20 ≈ 1 M subsets, n = 30 ≈ 1 B
- n > 64: array of words, word `i div 64`, bit `i mod 64`

ENUMERATION PATTERNS (PSEUDOCODE)

```
while s ≠ 0                          // visit each element of s
    visit(ctz(s))                    // index of lowest 1
    s ← s & (s - 1)                  // drop it

for mask ← 0 to (1 << n) - 1         // every subset of {0..n-1}
    process(mask)

sub ← m                              // every non-empty submask of m
while sub ≠ 0
    process(sub)
    sub ← (sub - 1) & m

function nextSameBits(x)             // Gosper's hack, x > 0
    c ← x & -x                       // lowest 1
    r ← x + c                        // carry ripples up
    return (((r ^ x) >> 2) div c) | r    // refill the low ones
```

CODE EXPLAINED

→ The submask loop visits 2^popcount(m) - 1 masks; over every m that's 3^n in total

→ Gosper walks k-subsets in order: `0111 → 1011 → 1101 → 1110 → 10011`

→ Bitmask DP state: (visited mask, last item). TSP runs in $O(2^n \cdot n^2)$, fine to n ≈ 20

SET OPERATIONS

SET	BITS
A ∪ B	`a | b`
A ∩ B	`a & b`
A − B	`a & ~b`
A Δ B	`a ^ b`
x ∈ A	`(a >> x) & 1`
|A|	`popcount(a)`
A ⊆ B	`(a & ~b) = 0`

BITSETS BEYOND 64 ELEMENTS

LANGUAGE	TYPE
by hand	`words[i >> 6] |= 1 << (i & 63)`
Java	`java.util.BitSet`
C++	`std::bitset<N>, std::vector<bool>`
Python	`int (unbounded)`
JavaScript	`Uint32Array of words`
Go	`[]uint64, math/big.Int`
Rust	`[u64; N], fixedbitset crate`

HOW FAST IT GROWS

N	10	15	20	25
2^n subsets	1,024	32,768	≈ 1.05 M	≈ 33.6 M
3^n submask visits	59,049	≈ 14.3 M	≈ 3.5 B	≈ 8.5×10^{11}

SUBMASKS OF M = 1011, IN LOOP ORDER

1	2	3	4	5	6	7
1011	1010	1001	1000	0011	0010	0001

⚠ COMMON MISTAKES

- The submask loop skips the empty set; handle 0 separately if needed
- `1 << n` with n = 32 on 32-bit ints
- 2^n enumeration past n ≈ 25: 33 M subsets × work each gets slow

✓ TIPS & BEST PRACTICES

- Map each element to a bit once and keep the map
- Precompute popcount per mask for DP tables
- Sparse huge sets: a hash set beats a bitmap

QUICK SUMMARY

- `a | b`, `a & b`, `a & ~b` — ∪, ∩, −
- `0 .. 2^n - 1` — all subsets
- `(sub - 1) & m` — next submask
- Gosper — next k-subset

Colors & Packed Data

SEVERAL VALUES, ONE INTEGER

Packing stores several small values side by side in one integer: shift each into its slot and OR them together; unpack with shift-then-mask. Colors are the everyday example. `0x3A7BD5` is three 8-bit channels in one 24-bit number, the same value CSS writes as `#3A7BD5`. The same trick builds hash keys from coordinates, squeezes colors into 16 bits, and stores fractions in integers.

NOTES

- Channel order is a convention: ARGB, RGBA, BGRA
- Alpha in the top byte makes a signed int negative

PACK & UNPACK (PSEUDOCODE)

```
c ← 0x3A7BD5                          // #3A7BD5
r ← (c >> 16) & 0xFF                  // 0x3A = 58
g ← (c >> 8) & 0xFF                   // 0x7B = 123
b ← c & 0xFF                          // 0xD5 = 213
packed ← (r << 16) | (g << 8) | b     // 0x3A7BD5 again
argb   ← (0xFF << 24) | packed        // opaque: 0xFF3A7BD5

// RGB565: 5 bits red, 6 green, 5 blue
rgb565 ← ((r >> 3) << 11) | ((g >> 2) << 5) | (b >> 3)   // 0x3BDA
r5 ← (rgb565 >> 11) & 0x1F
r8 ← (r5 << 3) | (r5 >> 2)            // 8 bits again: 57 (was 58)

// Two 32-bit ints in one 64-bit key
key ← (x << 32) | (y & 0xFFFFFFFF)    // mask y: negatives
x2  ← key >> 32
y2  ← key & 0xFFFFFFFF
```

CODE EXPLAINED

- → `& 0xFF` after the shift drops the channels above
- → `(r5 << 3) | (r5 >> 2)` copies high bits down so 31 → 255, not 248
- → Mask y or a negative y smears ones over x
- → **Morton (Z-order)** codes interleave x and y bits so nearby points get nearby keys

COMMON PACKED LAYOUTS

FORMAT	BIT LAYOUT	WHERE
ARGB8888	A 31–24 · R 23–16 · G 15–8 · B 7–0	Android, Java AWT, .NET
RGBA (bytes)	R, G, B, A in memory order	Canvas ImageData, WebGL, PNG
RGB565	R 15–11 · G 10–5 · B 4–0	small LCDs, embedded
Q16.16	integer 31–16 · fraction 15–0	DSP, fonts, old games
float32	sign 31 · exponent 30–23 · mantissa 22–0	IEEE-754

Q16.16 FIXED POINT

OP	EXPRESSION
to fixed	round(x × 65536)
to real	f / 65536
add	a + b
multiply	(a × b) >> 16 (64-bit product)
divide	(a << 16) / b
1.5	0x0001_8000

FLOAT32 BITS (READ WITH STD::BIT_CAST, FLOAT.FLOATTOINTBITS, STRUCT, DATAVIEW)

VALUE	BITS (HEX)	SIGN (31)	EXPONENT (30–23)	MANTISSA (22–0)
1.5	0x3FC00000	0	127 (2^0)	0x400000 (.5)
-2.0	0xC0000000	1	128 (2^1)	0
0.1	0x3DCCCCCD	0	123 (2^{-4})	0x4CCCCD (rounded)
+∞	0x7F800000	0	255	0

⚠ COMMON MISTAKES

- Java/C# int with alpha ≥ `0x80` is negative: `c >> 24` gives -1. Use `>>> 24` or `& 0xFF`
- Mixing RGBA and ARGB: channels come out rotated
- `x << 32` on a 32-bit x shifts by the width. Widen to 64 bits first

✓ TIPS & BEST PRACTICES

- Always `& 0xFF` after shifting a channel down
- Name the layout in code (`ARGB8888`); the bits can't tell you
- x86 BMI2 `pdep` / `pext` build Morton codes in one instruction

QUICK SUMMARY

- `(c >> 16) & 0xFF` — red from `0xRRGGBB`
- `(r << 16) | (g << 8) | b` — pack
- RGB565 — 5/6/5 bits in 16
- `(x << 32) | y` — two ints, one key

Sheet 13

Bytes, Endianness & Binary Formats

big / little-endian · network order · byte n · byte swap · UTF-8 bits

WHICH BYTE COMES FIRST?

Endianness is the order of a number's bytes in memory or in a file. **Little-endian** stores the least significant byte first (x86, ARM in practice). **Big-endian** stores the most significant byte first and is the **network byte order** of most protocols. Building a value from bytes with shifts works the same on every CPU, so it's the portable way to parse binary data.

NOTES

- `0x12345678` little: `78 56 34 12` ; big: `12 34 56 78`
- Only matters when bytes cross a boundary: files, sockets, raw memory

BYTES ↔ INTEGERS (PSEUDOCODE)

```
// Big-endian u32 from bytes b[0..3]
v ← (b[0] << 24) | (b[1] << 16) | (b[2] << 8) | b[3]
// Little-endian u32 from the same bytes
v ← b[0] | (b[1] << 8) | (b[2] << 16) | (b[3] << 24)

// Write v as big-endian bytes
for i ← 0 to 3
    out[i] ← (v >> (24 - 8 * i)) & 0xFF

byteN ← (v >> (8 * n)) & 0xFF          // n = 0 is the lowest byte

function bswap32(v)                    // reverse byte order
    return ((v & 0xFF) << 24) | ((v & 0xFF00) << 8)
         | ((v >> 8) & 0xFF00) | ((v >> 24) & 0xFF)
```

CODE EXPLAINED

- → Shifts act on values, not memory, so this code works on any CPU
- → Java/C# `byte` is signed: use `b[i] & 0xFF` before shifting
- → `b[0] << 24` can hit the sign bit in Java/JS; use long or `>>> 0`
- → Built-ins: `htonl` / `ntohl`, `ByteBuffer`, `DataView`, `struct`, `binary.BigEndian`

WHO USES WHICH ORDER

SYSTEM / FORMAT	ORDER
x86, x86-64, RISC-V	little
ARM (phones, Apple silicon)	little in practice
TCP/IP headers	big (network order)
JPEG, PNG, Java class files	big
BMP, WAV, ZIP, USB	little
UTF-16 text	either (BOM FE FF = big)

UTF-8 LEAD BITS

PATTERN	MEANING	TEST
0xxxxxxx	1-byte (ASCII)	b < 0x80
110xxxxx	starts 2 bytes	(b & 0xE0) == 0xC0
1110xxxx	starts 3 bytes	(b & 0xF0) == 0xE0
11110xxx	starts 4 bytes	(b & 0xF8) == 0xF0
10xxxxxx	continuation	(b & 0xC0) == 0x80

BYTES 12 34 56 78 READ BOTH WAYS

READ AS	BIG-ENDIAN	LITTLE-ENDIAN
u16 (b[0..1])	0x1234	0x3412
u32 (b[0..3])	0x12345678	0x78563412
i16 of FF FE	0xFFFE = −2	0xFEFF = −257
u8 (b[0])	0x12	0x12

HEADER BIT FIELDS: IPV4 (BYTES B0…B8)

FIELD	EXPRESSION
version	b0 >> 4 (4)
header length	(b0 & 0x0F) × 4 bytes
flags (DF, MF)	b6 >> 5
fragment offset	((b6 & 0x1F) << 8) | b7
TTL	b8

⚠ COMMON MISTAKES

- Casting a raw buffer to a struct and assuming file order = CPU order
- Skipping `& 0xFF` on signed bytes (Java, C#, C `char`)
- Swapping twice: once by hand, once by `ntohl`

✓ TIPS & BEST PRACTICES

- Python: `int.from_bytes(b, "big")`, `struct.unpack(">I", b)`
- JS: `new DataView(buf).getUint32(0)` is big-endian by default
- Java: `ByteBuffer.wrap(b)` then `.order(LITTLE_ENDIAN)`

QUICK SUMMARY

- little-endian — low byte first
- big-endian — high byte first (network)
- `(v >> 8n) & 0xFF` — byte n
- shifts — portable parsing

IP Addresses, Subnets & CIDR

IPv4 as 32-bit int · /24 mask `~0 << (32 - p)` · network `ip & mask` · broadcast `ip | ~mask` · in-subnet test · host count

SUBNETTING IS MASKING

An IPv4 address is a 32-bit number written as four bytes. A **CIDR prefix** like /24 says how many leading bits name the network; the rest name hosts. Turn the prefix into a **netmask** with one shift. Then AND gives the network address, OR with the inverted mask gives the broadcast address, and comparing masked values answers "is this IP in that subnet?"

NOTES

- 192.168.10.37 = `0xC0A80A25`
- /24 = 255.255.255.0 = `0xFFFFFF00`
- Prefix 0 would shift by 32: special-case it

SUBNET MATH (PSEUDOCODE)

```
function ipToInt(a, b, c, d)
    return (a << 24) | (b << 16) | (c << 8) | d

ALL   ← 0xFFFFFFFF
ip    ← ipToInt(192, 168, 10, 37)          // 0xC0A80A25
p     ← 26                                 // 192.168.10.37/26
mask  ← (ALL << (32 - p)) & ALL            // 255.255.255.192
net   ← ip & mask                          // 192.168.10.0
bcast ← net | (~mask & ALL)                // 192.168.10.63
hosts ← (1 << (32 - p)) - 2                // 62 usable

function inSubnet(addr, net, mask)
    return (addr & mask) = net

inSubnet(ipToInt(192, 168, 10, 70), net, mask)   // false
```

CODE EXPLAINED

- → `& ALL` keeps results in 32 bits where ints are wider (Python, Go `int`)
- → .70 is in the next /26 block (.64–.127)
- → /31 and /32 have no network/broadcast pair (RFC 3021)
- → Cisco **wildcard mask** = `~mask`: 0.0.0.63 for /26
- → IPv6: the same math on 128 bits (two 64-bit halves or a big int)

PREFIX → MASK → HOSTS

PREFIX	MASK	USABLE HOSTS
/8	255.0.0.0	16,777,214
/16	255.255.0.0	65,534
/24	255.255.255.0	254
/26	255.255.255.192	62
/28	255.255.255.240	14
/30	255.255.255.252	2

MASK OCTET VALUES

ONES	BITS	VALUE
1	1000_0000	128
2	1100_0000	192
3	1110_0000	224
4	1111_0000	240
5	1111_1000	248
6	1111_1100	252

RANGES TO KNOW

CIDR	MEANING
10.0.0.0/8	private
172.16.0.0/12	private
192.168.0.0/16	private
127.0.0.0/8	loopback
169.254.0.0/16	link-local
100.64.0.0/10	carrier NAT

192.168.10.37/26, STEP BY STEP

STEP	DOTTED	LAST OCTET (BITS)	EXPRESSION
address	192.168.10.37	0010_0101	`ip`
mask	255.255.255.192	1100_0000	`~0 << 6`
network	192.168.10.0	0000_0000	`ip & mask`
broadcast	192.168.10.63	0011_1111	`net | ~mask (hosts .1 - .62)`

⚠ COMMON MISTAKES

- Java ints make 192.x addresses negative; use `long` or unsigned compares
- JS `<<` gives negatives for 128.x and up; finish with `>>> 0`
- Matching strings (`startsWith("192.168.1")`) also matches 192.168.10.x

✓ TIPS & BEST PRACTICES

- Libraries: Python `ipaddress`, Go `net/netip`, .NET 8 `IPNetwork`
- Store IPv4 as unsigned 32-bit ints for fast sorting and range checks
- Same subnet? `(a & mask) == (b & mask)`

QUICK SUMMARY

- `~0 << (32 - p)` — mask for /p
- `ip & mask` — network address
- `net | ~mask` — broadcast
- `(ip & mask) == net` — membership

Hashing, Randomness & Branchless Tricks

SHIFT, XOR, MULTIPLY

Fast hash functions and random number generators are mostly shifts, XORs and multiplies. XOR-shifting folds high bits into low bits; multiplying by a large odd constant spreads low bits back up. Power-of-two table sizes turn `% cap` into `& (cap − 1)`. The same toolkit gives Gray codes and branch-free min, max and abs.

NOTES

- None of these are cryptographic
- Buckets use the low bits, so mix high bits down first
- Branchless code only helps on unpredictable branches. Measure

HASHING & PRNG BUILDING BLOCKS (PSEUDOCODE)

```
// Java HashMap-style spread before masking
index ← (h ^ (h >>> 16)) & (cap - 1)          // cap = 2^k

function fnv1a(bytes)                          // FNV-1a, 32-bit
    h ← 0x811C9DC5                             // offset basis
    for each b in bytes
        h ← ((h ^ b) * 0x01000193) & 0xFFFFFFFF   // × FNV prime
    return h

function xorshift32(s)                         // Marsaglia; s ≠ 0
    s ← s ^ ((s << 13) & 0xFFFFFFFF)
    s ← s ^ (s >>> 17)
    s ← s ^ ((s << 5) & 0xFFFFFFFF)
    return s

gray ← x ^ (x >> 1)                            // binary → Gray code
```

CODE EXPLAINED

- → A mask keeps only low bits, so `h >>> 16` mixes the high half in
- → FNV-1a XORs each byte in before multiplying (FNV-1 is the reverse)
- → xorshift32 has period $2^{32} - 1$ and never returns 0
- → Gray code: neighbors differ by one bit (0, 1, 3, 2, 6, 7, 5, 4); used in rotary encoders

BRANCHLESS (32-BIT SIGNED, ARITHMETIC >>)

GOAL	EXPRESSION
sign mask m	`x >> 31 (0 or -1)`
abs(x)	`(x ^ m) - m`
min(a, b)	`b ^ ((a ^ b) & -(a < b))`
max(a, b)	`a ^ ((a ^ b) & -(a < b))`
c ? a : b (c = 0/1)	`b ^ ((a ^ b) & -c)`
sign(x)	`(x > 0) - (x < 0)`
average, no overflow	`(a & b) + ((a ^ b) >> 1)`

CONSTANTS & RECIPES

NAME	RECIPE
Murmur3 fmix32	`h ^= h>>16; h *= 0x85EBCA6B; h ^= h>>13; h *= 0xC2B2AE35; h ^= h>>16`
Fibonacci hash	`(h × 0x9E3779B9) >> (32 - k)`
CRC-32	`shift + XOR with 0xEDB88320 (reflected)`
Gray decode	`g ^= g>>1; g ^= g>>2; … g ^= g>>16`
JS 32-bit multiply	`Math.imul(a, b)`

GRAY CODE: ONE BIT CHANGES PER STEP

N	0	1	2	3	4	5	6	7
binary	000	001	010	011	100	101	110	111
gray = n ^ (n >> 1)	000	001	011	010	110	111	101	100

⚠ COMMON MISTAKES

- `h % cap` with a negative Java hash gives a negative index. Use `& (cap - 1)`
- Seeding xorshift with 0: it stays 0 forever
- Using these hashes for passwords: use bcrypt or Argon2

✓ TIPS & BEST PRACTICES

- In Python, mask with `& 0xFFFFFFFF` after every multiply or left shift
- Use your language's tested hashes unless you need a specific algorithm
- Compilers already emit `cmov` for simple min/max; keep the readable version

QUICK SUMMARY

- `h & (cap - 1)` — bucket index
- `h ^ (h >>> 16)` — mix high bits down
- xorshift — fast non-crypto PRNG
- `x ^ (x >> 1)` — Gray code

Bitwise in Python

arbitrary-precision `int` · `~x == -x - 1` · no `>>>` · `& 0xFFFFFFFF` · `bin` / `format` / `int(s, 2)` · `bit_count` · `to_bytes` · `IntFlag`

UNLIMITED WIDTH, INFINITE SIGN BITS

Python ints are **arbitrary precision**: they never overflow, and negative numbers act as if they had infinitely many leading 1 bits. That keeps ~ and >> mathematically clean (`~x == -x - 1`, and >> floors), but there is no fixed width. When you need 32- or 64-bit behavior, for hashes, protocols or ports of C code, mask with `& 0xFFFFFFFF` yourself.

NOTES

- No `>>>`: mask first, then shift: `(x & 0xFFFFFFFF) >> n`
- `bool` is an int: `True & 1 == 1`
- On sets, `&` `|` `^` `-` mean ∩, ∪, Δ, difference

PYTHON BIT TOOLKIT

```
x = 0b1011_0010                          # 178
bin(x), hex(x), oct(x)                   # ('0b10110010', '0xb2', '0o262')
f"{x:08b} {x:#06x}"                      # '10110010 0x00b2'
int("1011_0010", 2), int("ff", 16)       # (178, 255)

~5, -7 >> 1, 1 << 64                     # (-6, -4, 18446744073709551616)
x.bit_length(), x.bit_count()            # (8, 4)   bit_count: 3.10+

def u32(v): return v & 0xFFFFFFFF        # wrap to unsigned 32-bit
u32(-1)                                  # 4294967295
u32(-16) >> 2                            # 1073741820  (Java's -16 >>> 2)
(0x12345678).to_bytes(4, "big")          # b'\x124Vx'
int.from_bytes(b"\x01\x00", "little") # 1
```

CODE EXPLAINED

→ `:08b` pads to 8 binary digits; `#` adds the `0x` / `0b` prefix

→ `bin(-5)` is `'-0b101'`. To see two's-complement bits: `bin(-5 & 0xFF)` → `'0b11111011'`

→ Signed view of 32 bits: `v - (1 << 32) if v & 0x80000000 else v`

→ `bit_count()` counts the magnitude: `(-1).bit_count()` is 1

→ `to_bytes` raises `OverflowError` if the value doesn't fit (or is negative without `signed=True`)

OPERATORS, PYTHON-STYLE

OPERATOR	PYTHON BEHAVIOR
`& | ^`	ints, bools and sets
`~x`	-x - 1, never an unsigned result
`x << n`	never overflows; the int grows
`x >> n`	floor(x / 2^n), sign-preserving
`>>>`	doesn't exist: mask, then >>
`&= |= ^= <<= >>=`	all compound forms exist

STANDARD LIBRARY & FRIENDS

TOOL	USE
`enum.IntFlag / Flag`	named flags: `Perm.R | Perm.W`
`struct`	fixed-width packing: `struct.pack("<I", v)`
`int.to_bytes / from_bytes`	endian conversion
`ipaddress`	`ip_network("10.0.0.0/8")`, `.netmask`
`numpy`	`np.uint32`, `np.unpackbits`, `np.bitwise_count` (2.0+)

NAMED FLAGS WITH INTFLAG

```
from enum import IntFlag

class Perm(IntFlag):
    R = 4
    W = 2
    X = 1

p = Perm.R | Perm.W          # <Perm.R|W: 6>
Perm.W in p                  # True
p & ~Perm.W                  # <Perm.R: 4>
```

PORTING 32-BIT CODE

```
def fnv1a_32(data: bytes) -> int:
    h = 0x811C9DC5
    for b in data:
        h ^= b
        h = (h * 0x01000193) & 0xFFFFFFFF  # stay 32-bit
    return h

hex(fnv1a_32(b"a"))          # '0xe40c292c'
```

⚠ COMMON MISTAKES

- Expecting `~0` to be `0xFFFFFFFF`. It's -1; use `~x & 0xFFFFFFFF`
- Porting C hashes without masking: values grow past 32 bits instead of wrapping
- `2 ^ 10` is 8 (XOR). Powers are `2 ** 10` or `1 << 10`

✓ TIPS & BEST PRACTICES

- `f"{x:032b}"` shows all 32 bits
- `IntFlag` members combine with `|` and print by name
- `bit_length()` gives floor($\log_2$ x) + 1 exactly; no float rounding

QUICK SUMMARY

- `int` — unbounded, never overflows
- `& 0xFFFFFFFF` — force 32-bit width
- `bin`, `format(x, "08b")`, `int(s, 2)` — print & parse
- `bit_length`, `bit_count` — length & popcount

Bitwise in Java

FIXED WIDTH, SIGNED, PROMOTED

Java integers are fixed-width two's complement: `byte` 8, `short` 16, `char` 16 (unsigned), `int` 32, `long` 64. Overflow wraps silently. Every bitwise operator promotes `byte`, `short` and `char` to `int` first, which causes most surprises. Java has >>> for logical right shift and a rich set of static helpers on `Integer` and `Long`.

NOTES

- Shift counts are masked: int `x << 33` is `x << 1`
- `(byte) 0xF0` is -16; `b & 0xFF` gives 240
- Unsigned helpers (Java 8+): `toUnsignedString`, `compareUnsigned`

JAVA BIT TOOLKIT

```
int x = 0b1011_0010;                    // 178
Integer.toBinaryString(x);              // "10110010"
Integer.toHexString(-1);                // "ffffffff"
Integer.parseInt("1011", 2);            // 11

int a = -16 >> 2;                       // -4
int b = -16 >>> 2;                      // 1073741820
int c = 1 << 33;                        // 2  (count masked to 1)
long d = 1L << 33;                      // 8589934592

byte raw = (byte) 0xF0;                 // -16
int u = raw & 0xFF;                     // 240

Integer.bitCount(x);                    // 4
Integer.numberOfTrailingZeros(x);       // 1
Integer.highestOneBit(x);               // 128
Integer.rotateLeft(x, 4);               // 2848
```

CODE EXPLAINED

→ `& 0xFF` undoes sign extension when a `byte` becomes an `int`

→ `>>>` on a `byte` shifts the promoted int: `(byte)-16 >>> 4` is 268435455

→ Bytes → int: `(b0 & 0xFF) << 24 | (b1 & 0xFF) << 16 | …`

→ `Long` mirrors `Integer`: `Long.bitCount`, `Long.numberOfLeadingZeros`

INTEGER / LONG HELPERS (X = 178)

METHOD	RESULT
`numberOfLeadingZeros(x)`	`24`
`lowestOneBit(x)`	`2`
`reverseBytes(0x12345678)`	`0x78563412`
`reverse(1)`	`0x80000000 (-2147483648)`
`toUnsignedString(-1)`	`"4294967295"`
`compress(x, m) (19+)`	`gather bits, like x86 pext`

TYPES & CLASSES FOR BITS

TYPE	USE
`java.util.BitSet`	growable bitset: `set`, `get`, `flip`, `cardinality`, `nextSetBit`
`EnumSet`	set of enum constants, stored as a long bitmask
`ByteBuffer`	`.order(ByteOrder.LITTLE_ENDIAN).getInt()`
`Math.floorMod`	non-negative modulo for negative hashes

BITSET & ENUMSET

```
BitSet seen = new BitSet();             // grows as needed
seen.set(3); seen.set(10);
seen.get(3);                            // true
seen.cardinality();                     // 2
seen.nextSetBit(4);                     // 10
EnumSet<Perm> p = EnumSet.of(Perm.R, Perm.W);
p.contains(Perm.W);                     // true
```

BYTES & UNSIGNED HELPERS

```
byte[] buf = {0x12, 0x34, 0x56, 0x78};
ByteBuffer.wrap(buf).getInt();          // 0x12345678 (big-endian
ByteBuffer.wrap(buf)
    .order(ByteOrder.LITTLE_ENDIAN)
    .getInt();                          // 0x78563412
long u = Integer.toUnsignedLong(-1);    // 4294967295
Integer.compareUnsigned(-1, 1);         // 1: -1 is the largest
```

⚠ COMMON MISTAKES

- `1 << 40` without `L` gives 256
- `long m = 0xFFFFFFFF;` is -1L (sign-extended int). Write `0xFFFFFFFFL`
- `x & MASK == 0` doesn't compile; it means `x & (MASK == 0)`. Parenthesize

✓ TIPS & BEST PRACTICES

- Add `L` to any constant you'll shift by 31 or more
- `toBinaryString` drops leading zeros; pad with `String.format`
- Prefer `EnumSet` over int flags in public APIs

QUICK SUMMARY

- `>>>` — logical right shift
- `b & 0xFF` — unsigned byte
- `1L << n` — 64-bit shifts
- `Integer` / `Long` — bitCount, rotate, reverse

Bitwise in JavaScript

FLOATS THAT BECOME 32-BIT INTS

JavaScript numbers are 64-bit floats, but every bitwise operator first converts its operands to **32-bit signed integers**, does the work, and returns a 32-bit result. >>> is the exception: it returns an **unsigned** 32-bit value. For anything wider than 32 bits use **BigInt** (123n), which supports every bitwise operator except >>>.

NOTES

- Results ≥ 2^{31} come back negative: `0x80000000 | 0` is -2147483648
- `x | 0` truncates toward zero, but only inside 32-bit range
- Plain numbers are exact only up to $2^{53} - 1$

JAVASCRIPT BIT TOOLKIT

```
const x = 0b1011_0010;                  // 178
x.toString(2);                          // "10110010"
x.toString(2).padStart(16, "0");        // "0000000010110010"
parseInt("ff", 16);                     // 255

1 << 31;                                // -2147483648
(1 << 31) >>> 0;                        // 2147483648
-16 >> 2;                               // -4
-16 >>> 2;                              // 1073741820
1 << 32;                                // 1   (count masked to 0)
2 ** 32 | 0;                            // 0   (ToInt32 wraps)
Math.clz32(1);                          // 31
Math.imul(0xFFFFFFFF, 5);               // -5  (32-bit multiply)

1n << 64n;                              // 18446744073709551616n
BigInt.asUintN(64, -1n);                // 18446744073709551615n
```

CODE EXPLAINED

- → `>>> 0` reinterprets a result as unsigned 32-bit (colors, hashes)
- → BigInt and Number don't mix: `1n << 3` throws `TypeError`; convert with `BigInt(n)`
- → `BigInt.asUintN(w, b)` / `asIntN` wrap to w bits like a fixed-width type
- → Typed arrays (`Uint8Array`, `Uint32Array`) wrap on write; `DataView` picks endianness

TOINT32 IN ACTION

EXPRESSION	RESULT
`3.7 | 0`	3
`-3.7 | 0`	-3
`2147483648 | 0`	-2147483648
`3_000_000_000 | 0`	-1294967296
`NaN | 0`	0
`~~"5.9"`	5

NUMBER VS BIGINT

	NUMBER	BIGINT
width for bit ops	32-bit	unbounded
literal	0xFF	0xFFn
>>>	yes	TypeError
fixed width	automatic	BigInt.asUintN(64, b)
leading zeros	Math.clz32	none (loop)
mixing types	—	TypeError

TYPESCRIPT FLAG ENUM

```
enum Perm { None = 0, R = 1 << 0, W = 1 << 1, X = 1 << 2 }

let p: Perm = Perm.R | Perm.W;        // 3
const canWrite = (p & Perm.W) !== 0;  // true
p &= ~Perm.W;                         // remove W → 1
const all = Perm.R | Perm.W | Perm.X; // 7
```

BINARY DATA: DATAVIEW & TYPED ARRAYS

```
const buf = new Uint8Array([0x12, 0x34, 0x56, 0x78]).buffer;
const dv = new DataView(buf);
dv.getUint32(0).toString(16);         // "12345678" (big-endian)
dv.getUint32(0, true).toString(16);   // "78563412" (little)
new Uint8Array([300])[0];             // 44 (wraps mod 256)
```

⚠ COMMON MISTAKES

- `x | 0` as floor for big values: `3_000_000_000 | 0` is negative. Use `Math.trunc`
- `1 << 40` to reach 64 bits gives 256. Use `1n << 40n`
- Comparing hashes or colors without `>>> 0`: negatives sort and print wrong

✓ TIPS & BEST PRACTICES

- View 32 bits of a negative: `(n >>> 0).toString(2)`
- TS flags: `enum Perm { R = 1 << 0, W = 1 << 1 }`, then `Perm.R | Perm.W`
- Parse binary data with `DataView` / `Uint8Array`, not strings

QUICK SUMMARY

- bit ops — on 32-bit signed ints
- `>>> 0` — as unsigned
- `BigInt` — 64-bit and beyond
- `Math.clz32`, `Math.imul` — 32-bit helpers

Bitwise in C & C++

CLOSE TO THE METAL, FULL OF TRAPS

C and C++ map bit operations straight onto the hardware. Use **unsigned fixed-width types** (`uint8_t` … `uint64_t`) for bit work: unsigned overflow wraps by definition, while signed overflow is **undefined behavior**. Types smaller than `int` are promoted to `int` before any operator runs, and shifting by a negative count or by ≥ the width is undefined.

NOTES

- `>>` of a negative: implementation-defined in C, arithmetic in C++20
- Wide literals: `1u`, `1ull`, `UINT64_C(1)`
- `0b` literals: C++14, C23. Separators: `0b1011'0010` (C++14, C23)

C++ BIT TOOLKIT

```cpp
#include <bit>      // C++20; also <cstdint>, <bitset>

uint32_t x = 0b1011'0010;                  // 178
uint32_t m = 1u << 31;                     // fine: unsigned
uint64_t big = 1ull << 40;                 // 1 << 40 would be UB
uint8_t b = 0xF0;
auto y = ~b;                               // int -241 (promoted!)
uint8_t z = static_cast<uint8_t>(~b);      // 0x0F

std::popcount(x);                          // 4
std::countr_zero(x);                       // 1
std::bit_width(x);                         // 8
std::rotl(x, 4);                           // 2848
std::has_single_bit(64u);                  // true
std::bitset<8>(x).to_string();             // "10110010"
auto bits = std::bit_cast<uint32_t>(1.5f); // 0x3FC00000
```

CODE EXPLAINED

→ **Promotion**: `uint8_t` / `uint16_t` become `int`, so `~b` and `b << 24` run signed

→ GCC/Clang: `__builtin_popcount`, `__builtin_ctz`, `__builtin_clz` (undefined at 0); `ll` suffix for 64-bit

→ MSVC: `__popcnt`, `_BitScanForward`, `_byteswap_ulong`

→ C23 `<stdbit.h>`: `stdc_count_ones`, `stdc_leading_zeros`, `stdc_bit_width`

DEFINED OR NOT?

EXPRESSION	STATUS
`INT_MAX + 1`	UB (signed overflow)
`UINT_MAX + 1u`	defined: wraps to 0
`x << 32 (32-bit x)`	UB: count ≥ width
`x << -1`	UB: negative count
`-8 >> 1`	impl.-defined in C; -4 in C++20
`1 << 31 (int)`	UB in C; defined in C++20

FIELDS & CONTAINERS

TOOL	EXAMPLE	NOTE
`bit field`	`unsigned g : 6;`	layout is compiler-specific
`std::bitset<N>`	`.test .set .flip .count`	fixed size
`std::vector<bool>`	`v[i] = true`	packed; proxy refs
`std::byte (C++17)`	`std::byte{0xF0} >> 4`	bit ops only
`std::byteswap`	`std::byteswap(x)`	C++23; `std::endian` C++20

C: BIT FIELDS & BUILTINS

```c
#include <stdint.h>
/* bit fields: the order in memory varies by compiler */
struct Rgb565 { unsigned b : 5, g : 6, r : 5; };
uint32_t x = 178;
__builtin_popcount(x);       /* 4  (GCC/Clang) */
__builtin_ctz(x);            /* 1  (x must be != 0) */
__builtin_clz(x);            /* 24 */
```

C23: <STDBIT.H> & PRINTF %B

```c
#include <stdbit.h>          /* C23 */
stdc_count_ones(x);          /* 4 */
stdc_trailing_zeros(x);      /* 1 */
stdc_bit_width(x);           /* 8 */
printf("%b\n", x);           /* 10110010 (C23) */
printf("%08x\n", x);         /* 000000b2 */
```

⚠ COMMON MISTAKES

- `1 << 40` with an `int` literal. Write `1ull << 40`
- `char` signedness varies: `(char)0xF0 >> 4` may be -1 or 15
- Mixing signed and unsigned: `-1 < 0u` is false

✓ TIPS & BEST PRACTICES

- Build with `-Wall -Wextra -Wconversion -fsanitize=undefined`
- Use `uint8_t` / `std::byte` for raw bytes, not `char`
- Prefer `<bit>` over compiler builtins for portable code

QUICK SUMMARY

- `uint32_t` — unsigned, wraps safely
- promotion — small types become `int`
- shift ≥ width — UB
- `<bit>` — popcount, rotl, bit_cast

Bitwise in C# & Go

TWO LANGUAGES WITH REAL UNSIGNED TYPES

C# works like Java (two's complement, wraps by default) but adds unsigned types (`byte` is unsigned, `sbyte` signed, plus `uint`, `ulong`), >>> since C# 11, and `[Flags]` enums. **Go** also has unsigned types, writes NOT as unary ^x, adds &^ (AND NOT, "bit clear"), and keeps helpers in `math/bits`.

NOTES

- C#: wraps unless `checked`; shift counts masked
- Go: shifting by ≥ width gives 0 (-1 for a negative `>>`)
- Go needs explicit conversions: `uint32(b) << 24`

C#

```
uint x = 0b1011_0010;                  // 178
Convert.ToString(x, 2);                // "10110010"
x.ToString("X8");                      // "000000B2"
int a = -16 >>> 2;                     // 1073741820
int b = 1 << 33;                       // 2 (count masked)
BitOperations.PopCount(x);             // 4
BitOperations.TrailingZeroCount(x);    // 1
BitOperations.RotateLeft(x, 4);        // 2848
uint.IsPow2(64u);                      // true (.NET 7+)

[Flags] enum Perm { None = 0, R = 1, W = 2, X = 4 }
var p = Perm.R | Perm.W;
p.HasFlag(Perm.W);                     // true
p.ToString();                          // "R, W"
```

GO

```
x := uint32(0b1011_0010)           // 178
fmt.Sprintf("%08b %#x", x, x)      // "10110010 0xb2"
strconv.ParseUint("ff", 16, 32)    // 255, nil
^x                                 // NOT: 4294967117
x &^ 0b11                          // AND NOT: 176
bits.OnesCount32(x)                // 4
bits.TrailingZeros32(x)            // 1
bits.Len32(x)                      // 8
bits.RotateLeft32(x, 4)            // 2848 (k < 0: right)
binary.BigEndian.Uint32(buf)       // bytes → uint32

const (
    R = 1 << iota                  // 1, then W = 2, X = 4
    W
    X
)
```

C# REFERENCE

FEATURE	C#
unsigned types	byte ushort uint ulong (sbyte is signed)
overflow	wraps; checked throws OverflowException
bit helpers	System.Numerics.BitOperations, uint.PopCount
collections	BitArray, BitVector32
endianness	BinaryPrimitives.ReadUInt32BigEndian

GO REFERENCE

FEATURE	GO
NOT / AND NOT	^x / x &^ y
shift count	any integer type; negative panics
math/bits	OnesCount, LeadingZeros, Len, Reverse, Add64
encoding/binary	BigEndian, LittleEndian, PutUint32
overflow	wraps; never undefined

C#: BITARRAY, ENDIANNESS, CHECKED

```
var bits = new BitArray(16) { [3] = true };
bits.Get(3);                                  // true
Span<byte> buf = stackalloc byte[4];
BinaryPrimitives.WriteUInt32BigEndian(buf, 0x12345678);
BinaryPrimitives.ReadUInt32LittleEndian(buf); // 0x78563412
checked { int n = int.MaxValue; n++; }  // OverflowException
```

GO: A BITSET TYPE

```
type Bitset []uint64
func (s Bitset) Has(i int) bool {
    return s[i>>6]&(1<<(i&63)) != 0
}
func (s Bitset) Set(i int) { s[i>>6] |= 1 << (i & 63) }
s := make(Bitset, 2)          // 128 bits
s.Set(70); s.Has(70)          // true
```

⚠ COMMON MISTAKES

- C#: `byte c = a & b;` won't compile; byte ops return `int`. Cast back
- Go: `~x` doesn't compile as NOT; write `^x`
- Go: `b << 24` on a `byte` stays a byte (0). Convert first

✓ TIPS & BEST PRACTICES

- C#: give `[Flags]` enums powers of two and a `None = 0`
- C#: `BinaryPrimitives` parses bytes without allocations
- Go: `%b`, `%08b`, `%#x` print bits; `bits.Add64` / `Mul64` do carry math

QUICK SUMMARY

- C# `>>>`, `[Flags]`, `BitOperations`
- Go `^x` — NOT; `&^` — AND NOT
- Go `math/bits` — count, rotate, reverse
- both — wrap on overflow, no UB

Bitwise in Rust, Kotlin, Ruby & SQL

SAME IDEAS, DIFFERENT SPELLINGS

Rust writes NOT as `!` and panics on shift overflow in debug builds. **Kotlin** has no bitwise symbols: the operators are infix functions (`and`, `or`, `xor`, `inv()`, `shl`, `shr`, `ushr`). **Ruby** integers are unbounded like Python's and can read bit i with `x[i]`. **SQL** dialects agree on & and |, but not on XOR.

NOTES

- Rust: `wrapping_*`, `checked_*`, `overflowing_*` make overflow explicit
- Kotlin infix binds looser than `+` / `-` but tighter than `==`
- PostgreSQL XOR is `#`; MySQL and SQL Server use `^`

RUST

```rust
let x: u32 = 0b1011_0010;            // 178
format!("{x:08b} {x:#x}");           // "10110010 0xb2"
u32::from_str_radix("ff", 16);       // Ok(255)
!x;                                  // NOT: 4294967117
x.count_ones();                      // 4
x.trailing_zeros();                  // 1
x.rotate_left(4);                    // 2848
x.swap_bytes();                      // 0xB2000000
x.to_be_bytes();                     // [0, 0, 0, 178]
1u32.wrapping_shl(33);               // 2 (count masked)
1u32.checked_shl(33);                // None
-16i32 >> 2;                         // -4
(-16i32 as u32) >> 2;                // 1073741820
```

KOTLIN

```kotlin
val x = 0b1011_0010                  // 178
x.toString(2)                        // "10110010"
"ff".toInt(16)                       // 255
x and 0x0F                           // 2
x or 1                               // 179
x xor 0xFF                           // 77
x.inv()                              // -179
-16 shr 2                            // -4
-16 ushr 2                           // 1073741820
1L shl 40                            // 1099511627776
x.countOneBits()                     // 4
x.takeLowestOneBit()                 // 2
0xFFu.toUByte()                      // 255 (unsigned types)
```

RUBY

```ruby
x = 0b1011_0010                      # 178
x.to_s(2)                            # "10110010"
"ff".to_i(16)                        # 255
[x[1], x[0]]                         # [1, 0]  read bit i
x.bit_length                         # 8
x.to_s(2).count("1")                 # 4  (popcount)
x.anybits?(0b11)                     # true   (2.5+)
x.allbits?(0b11)                     # false
~5                                   # -6
[0x12345678].pack("N")               # "\x124Vx" big-endian
```

SQL

```sql
-- PostgreSQL
SELECT 12 & 10, 12 | 10, 12 # 10, ~12;  -- 8, 14, 6, -13
SELECT 1 << 4, bit_count(B'1011');      -- 16, 3 (PG 14+)
SELECT 178::bit(8);                     -- 10110010
-- MySQL / MariaDB
SELECT 12 ^ 10, BIT_COUNT(178);         -- 6, 4
SELECT BIN(178);                        -- '10110010'
-- SQL Server
SELECT 12 ^ 10, 13 & ~1;                -- 6, 12
-- Flags column: rows with bit 2 set
SELECT * FROM files WHERE (perms & 4) <> 0;
```

AT A GLANCE

LANGUAGE	NOT	LOGICAL >>	OVERFLOW	POPCOUNT
Rust	`!x`	`>>` on u32/u64	panics in debug, wraps in release	`count_ones()`
Kotlin	`x.inv()`	`ushr`	wraps	`countOneBits()`
Ruby	`~x`	— (unbounded)	never (Integer grows)	`to_s(2).count("1")`
PostgreSQL	`~x`	—	error: integer out of range	`bit_count()` (14+)

⚠ COMMON MISTAKES

- Rust: `x << 32` on u32 panics in debug. Use `checked_shl` / `wrapping_shl`
- Kotlin: `1 shl n - 1` is `1 shl (n - 1)`
- PostgreSQL `^` is power: `2 ^ 3` = 8; MySQL `2 ^ 3` = 1

✓ TIPS & BEST PRACTICES

- Rust: `u32::from_be_bytes` for parsing; `bitflags` crate for flag sets
- Kotlin: `UInt` / `ULong` for unsigned math (`0xFFu`)
- SQL: wrap flag tests in a view with named boolean columns

QUICK SUMMARY

- Rust `!x`, `count_ones`, `wrapping_shl`
- Kotlin `and` `or` `xor` `inv` `shl` `ushr`
- Ruby `x[i]`, `to_s(2)`, unbounded ints
- SQL `&` `|`; XOR `#` (PG) / `^` (MySQL)

Sheet 22

Bitwise on the Command Line

bash / zsh `$(( a & b ))` · `2#1010` / `16#ff` · zsh `[##2]` · `printf %x` · `bc obase=2` · PowerShell `-band` `-bor` `-bxor` `-shl` · `chmod` · `xxd -b`

BIT MATH WITHOUT OPENING AN EDITOR

Shells do integer arithmetic inside `$(( ))` with the C operators: bash and zsh use 64-bit signed integers and accept any base with `base#digits`. **PowerShell** spells the operators as words (`-band`, `-bor`, `-bxor`, `-bnot`, `-shl`, `-shr`) and can call .NET helpers directly. For conversions, `printf`, `bc` and a Python one-liner cover the rest.

NOTES

- No `>>>` in shells; `>>` is arithmetic
- `(( x & 4 ))` returns exit status 0 when the bit is set
- `[ ]` / `test` can't do bit math; use `(( ))`

BASH / ZSH

```
echo $(( 12 & 10 )) $(( 12 | 10 )) $(( 12 ^ 10 ))  # 8 14 6
echo $(( ~12 )) $(( 1 << 10 )) $(( -16 >> 2 ))     # -13 1024 -4
echo $(( 2#1011 )) $(( 16#ff )) $(( 0x1F ))        # 11 255 31
echo $(( 017 )) $(( 10#017 ))                      # 15 17
printf '%x %o %X\n' 255 8 3054                     # ff 10 BEE
printf '%d\n' 0x1F                                 # 31
echo 'obase=2; 178' | bc                           # 10110010
(( flags & 4 )) && echo "bit 2 set"
mask=$(( (1 << 6) - 1 ))                           # 63

# zsh only: print in a base
echo $(( [##2] 178 )) $(( [##16] 255 ))            # 10110010 FF
```

POWERSHELL

```
12 -band 10                          # 8
12 -bor 10                           # 14
12 -bxor 10                          # 6
-bnot 12                             # -13
1 -shl 10                            # 1024
-16 -shr 2                           # -4
[uint32]4294967280 -shr 2            # 1073741820
[Convert]::ToString(178, 2)          # 10110010
[Convert]::ToInt32('ff', 16)         # 255
'{0:X8}' -f 178                      # 000000B2
0b1011                               # 11 (PowerShell 7+)
```

HANDY TOOLS

COMMAND	DOES
`echo 'ibase=2; 1011' | bc`	binary → decimal (11)
`python3 -c 'print(f"{178:08b}")'`	any conversion, one line
`xxd -b file / xxd -p file`	dump bits / plain hex
`od -An -tx1 file`	hex bytes
`ipcalc 192.168.10.37/26`	network, broadcast, host range
`stat -c %a f (Linux) / stat -f %Lp f (macOS)`	octal mode of a file

POWERSHELL EXTRAS

TASK	EXPRESSION
popcount	`[Numerics.BitOperations]::PopCount([uint]178)`
pad binary	`[Convert]::ToString(5, 2).PadLeft(8, '0')`
test a flag	`($attrs -band 2) -ne 0`
enum flags	`[IO.FileAttributes]'Hidden, ReadOnly'`
hex → int	`[int]'0xFF'`

BASH: PRINT N BITS

```
tobin() {                          # tobin VALUE [WIDTH]
  local n=$1 w=${2:-8} s=""
  for (( i = w - 1; i >= 0; i-- )); do
    s+=$(( (n >> i) & 1 ))
  done
  echo "$s"
}
tobin 178                          # 10110010
tobin 5 4                          # 0101
```

BASH: PERMISSION BITS

```
mode=$(stat -c %a file.txt)                # e.g. 644 (Linux)
(( 8#$mode & 8#200 )) && echo "owner can write"
printf '%04o\n' $(( 8#666 & ~8#027 ))      # 0640
umask 027                                  # files 640, dirs 750
```

⚠ COMMON MISTAKES

- `$(( 017 ))` is 15: a leading zero means octal
- `$(( 08 ))` is an error (bad octal). Use `10#08`
- Reading `^` as power: bash's power operator is `2 ** 10`

✓ TIPS & BEST PRACTICES

- zsh: `[##2]` prints binary, `[##16]` prints hex
- `chmod 754 f` sets `rwxr-xr--`; `umask` shows the mask
- `jq` has no bitwise operators; pipe numbers to the shell or Python

QUICK SUMMARY

- `$(( ))` — C operators, 64-bit signed
- `base#digits` — any base in
- `printf '%x'`, `bc obase=2` — bases out
- PowerShell `-band` `-bor` `-bxor` `-shl`

Quick Reference & Language Matrix

OPERATORS (03–04)

EXPR	MEANS
`a & b`	1 where both
`a | b`	1 where either
`a ^ b`	1 where different
`~a`	flip all (-a - 1)
`a << n`	× 2^n, zeros in
`a >> n`	÷ 2^n floor, sign in
`a >>> n`	zeros in (logical)

SINGLE BIT I (05)

EXPR	MEANS
`(x >> i) & 1`	test
`x | (1 << i)`	set
`x & ~(1 << i)`	clear
`x ^ (1 << i)`	toggle
`(x & m) == m`	all of m set
`(x & m) != 0`	any of m set
`x & 1`	odd?

MASKS & FIELDS (06, 13)

EXPR	MEANS
`(1 << n) - 1`	n low ones
`low(w) << lo`	bits lo..lo+w-1
`(x >> lo) & low(w)`	read field
`(x & ~m) | (v << lo)`	write field
`(x >> 8n) & 0xFF`	byte n
`(x ^ m) - m`	sign-extend
`~0`	all ones

LOWEST BIT & 2^n (07)

EXPR	MEANS
`x & (x - 1)`	clear lowest 1
`x & -x`	isolate lowest 1
`x | (x + 1)`	set lowest 0
`(x & (x - 1)) == 0`	power of 2 (x ≠ 0)
`x & (2^n - 1)`	x mod 2^n
`(x + a - 1) & ~(a - 1)`	align up to a

COUNTING & SETS (08, 11)

EXPR	MEANS
`x &= x - 1; c++`	popcount loop
`popcount((x & -x) - 1)`	trailing zeros
`w - clz(x)`	bit length
`popcount(a ^ b)`	Hamming distance
`(s - 1) & m`	next submask
`a & ~b`	set difference

XOR & MISC (09, 14, 15)

EXPR	MEANS
`XOR of all`	the unpaired value
`x ^ (x >> 1)`	Gray code
`h & (cap - 1)`	bucket (cap = 2^n)
`h ^ (h >>> 16)`	mix high bits
`~0 << (32 - p)`	netmask for /p
`(a & b) + ((a ^ b) >> 1)`	average, no overflow

LANGUAGE MATRIX (16–22)

LANGUAGE	INT WIDTH	NOT	LOGICAL >>	POPCOUNT	BINARY LITERAL	PRINT BINARY
Python	unbounded	`~x`	`mask, then >>`	`x.bit_count()`	`0b101`	`f"{x:b}"`
Java	32 / 64	`~x`	`>>>`	`Integer.bitCount(x)`	`0b101`	`Integer.toBinaryString(x)`
JavaScript	32 (BigInt: any)	`~x`	`>>>`	—	`0b101`	`x.toString(2)`
C / C++	8–64	`~x`	`>> on unsigned`	`std::popcount(x)`	`0b101`	`printf("%b") C23 · std::bitset`
C#	8–64	`~x`	`>>> (C# 11)`	`BitOperations.PopCount`	`0b101`	`Convert.ToString(x, 2)`
Go	8–64	`^x`	`>> on uint`	`bits.OnesCount(x)`	`0b101`	`fmt.Sprintf("%b", x)`
Rust	8–128	`!x`	`>> on u32`	`x.count_ones()`	`0b101`	`format!("{:b}", x)`
Kotlin	8–64	`x.inv()`	`ushr`	`x.countOneBits()`	`0b101`	`x.toString(2)`
Ruby	unbounded	`~x`	—	`x.to_s(2).count("1")`	`0b101`	`x.to_s(2)`
bash / zsh	64 signed	`~x`	—	—	`2#101`	`bc obase=2 · zsh [##2]`
PowerShell	32 / 64	`-bnot x`	`-shr on [uint]`	`BitOperations`	`0b101 (7+)`	`[Convert]::ToString(x, 2)`

⚠ COMMON MISTAKES

- `x & m == 0` in C, Java, JS, C#: parenthesize
- `1 << 40` with a 32-bit 1: use a 64-bit one
- Signed bytes: `b & 0xFF` before shifting
- Shift ≥ width: undefined or masked

✓ TIPS & BEST PRACTICES

- Use unsigned types for bit patterns
- Write masks in hex: `0xFF00`
- Prefer built-ins over hand-written loops
- Name your flags and fields

QUICK SUMMARY

- `&` — keep · `|` — set · `^` — flip
- `~` — invert · `<<` `>>` — move
- `x & (x - 1)` — the most useful trick
- Sheet numbers in titles point to details